INTELLIGENT UNIVERSITY EXAM HALL SEATING ALLOCATOR

Project Proposal

Submitted by:

Ahsan Raza, SP25-BDS-003

Muzzamil Ghaffar, SP25-BDS-034

Muhammad Irshad Khan, SP25-BDS-025

Sadaqat Hussain, SP25-BDS-040

Department of Computer Science, CUI, Islamabad.

Course: CSC262: Artificial Intelligence - I (Fall 2026)

Date: October 27, 2026

Project Category: Artificial Intelligence - Constraint Satisfaction Problem (CSP) / Search

Table of Contents

Abstract 2

1. Introduction 2

2. Proposed System 2

3. Advantages/Benefits of Proposed System 3

4. Scope 3

5. Modules 3

5.1 Module 1: Data Ingestion & Preprocessing Module 3

5.2 Module 2: CSP Solver & Constraint Engine Module 3

5.3 Module 3: Room & Infrastructure Manager Module 3

5.4 Module 4: Visualization & Export Module 4

6. System Limitations/Constraints 4

7. Tools and Technologies 4

8. Conclusion 4

9. References 4

Abstract

Allocating examination seating for universities handling hundreds or thousands of students is a complex combinatorial problem that is traditionally prone to human error, administrative bottlenecks, and academic integrity risks. This project presents an automated, AI-driven Intelligent University Exam Hall Seating Allocator modeled as a Constraint Satisfaction Problem (CSP) and solved via Backtracking Search. The system ingests student rosters, prioritizes hard accessibility constraints (ADA compliance), and enforces an anti-cheating isolation rule by preventing students of the same degree section or the same course paper from occupying orthogonally adjacent seats.

Designed and implemented in Python, the system processes and validates a multi-room seating allocation for 50 students across three examination halls, demonstrating correct, repeatable results and adherence to institutional academic standards.

1. Introduction

The purpose of this project proposal document is to establish the design, architectural framework, and implementation blueprint for an automated exam hall seating allocation system. In modern universities, academic scheduling and invigilation planning require careful attention to physical constraints, institutional capacities, and academic honesty. Traditional manual seating arrangements are tedious, error-prone, and inefficient, and they often fail to detect section clusters or students sitting the same paper side by side. Furthermore, failing to accommodate students with physical impairments violates accessibility expectations.

This software addresses the real-world problem of examination hall cheating vulnerabilities and accessibility oversights by replacing manual allocation with an automated AI approach. By formalizing seating arrangement as a constraint satisfaction problem, the system guarantees that no two students taking the same course paper, or belonging to the same degree section, sit directly next to each other (left, right, front or back). Through this development, key technical skills are acquired, including algorithmic problem modeling, state-space search, backtracking recursion, constraint checking, and structured data handling using the Python library Pandas.

2. Proposed System

The proposed system addresses the complexities of institutional examination seating by applying core artificial intelligence and search techniques covered in coursework. Instead of relying on manual sorting, the application models the exam hall layout as a Constraint Satisfaction Problem (CSP). Students represent the variables requiring assignment, physical hall seats serve as the domains, and institutional rules act as the governing constraints.

To achieve conflict-free allocation, the system first separates students with documented mobility requirements (ADA compliance) and places them first, into accessible seating, before regular students are scheduled. The core CSP engine then executes a recursive backtracking search. For each student it tries the available seats in order, and a safety-validation function checks both the hard accessibility requirement and the anti-cheating isolation rule. If a branch of assignments leads to a dead end, the algorithm backtracks, undoes the last assignment, and explores the next alternative. Finally, the system compiles the verified allocation into a structured, tabular report ready for invigilation staff.

3. Advantages/Benefits of Proposed System

1. Elimination of Human Error: automates seating generation, removing manual calculation mistakes.

2. Enhanced Academic Integrity: enforces orthogonal isolation for identical degree sections and course papers, reducing cheating opportunities.

3. Guaranteed ADA Accessibility: places students with mobility needs first and only in accessible seats.

4. Fast Execution: produces a valid multi-room layout for dozens of students in a fraction of a second.

5. Scalable Multi-Room Design: supports any number of halls with different row and column configurations.

6. Actionable Tabular Reporting: generates a clean, sorted seating table ready for printing and distribution to invigilators.

4. Scope

The scope of this project covers the design, implementation, and evaluation of an automated Python-based Constraint Satisfaction Problem solver tailored to university examination seating. The system takes a student roster containing student identifiers, names, degree sections, course codes, and ADA accommodation flags. It manages multiple halls with configurable row and column dimensions, runs a backtracking search with strict orthogonal adjacency separation, and prints a formatted allocation report.

The boundaries of the solution are as follows. The system handles spatial isolation, room capacities, and accessibility rules. It does not perform optical character recognition (OCR) student card scanning, real-time biometric attendance tracking, or faculty invigilation duty rostering. These external administrative modules fall outside the current project scope.

5. Modules

5.1 Module 1: Data Ingestion & Preprocessing Module

This module is responsible for loading and preparing the raw student roster. It converts the roster DataFrame into a list of student records and separates students requiring ADA accommodations from standard students so that the ADA group is handled with priority during allocation. In the demonstration, the roster is generated synthetically for 50 students with a fixed random seed so the results are reproducible.

5.2 Module 2: CSP Solver & Constraint Engine Module

Acting as the core intelligence of the application, this module implements the backtracking search and the constraint validation logic. It evaluates each student variable against the available seat domain, enforcing the hard ADA requirement and the orthogonal anti-cheating rule. When a dead end occurs, it unwinds the recursion and restores the previous state.

5.3 Module 3: Room & Infrastructure Manager Module

This module models the physical examination venues: multiple halls, their capacities, row and column dimensions, and accessibility flags. It generates the global pool of physical seats, assigning each seat an identifier, coordinates, and an ADA-compliance flag.

5.4 Module 4: Visualization & Export Module

This module takes the verified allocation dictionary produced by the CSP engine and compiles it into a human-readable report using Pandas. The final table lists the seat, student ID, name, section, course, and ADA status for every student, sorted by seat.

6. System Limitations/Constraints

- The system assumes static room grid layouts with uniform row and column dimensions per hall.
- Backtracking search can take exponentially long in dense, over-constrained scenarios where the number of seats closely matches the number of students, leaving little room to avoid conflicts.

7. Tools and Technologies

The software and technologies used in the implementation are listed in Table 1. The entire system is implemented in Python.

Table 1: Tools and Technologies for Project

Category	Tool / Technology	Version	Rationale
IDE	Visual Studio Code	Latest	Lightweight, robust code editing, running and debugging environment
Programming Language	Python	3.11	Core language for implementing the object-oriented CSP solver
Data Processing Library	Pandas	2.0+	Holds the student roster as a DataFrame and produces the tabular seating report
Randomization Utility	random (Python standard library)	Built-in	Generates a reproducible synthetic dataset of 50 students (seed 42)

8. Conclusion

The Intelligent University Exam Hall Seating Allocator demonstrates the practical application of Artificial Intelligence principles to an institutional scheduling problem. By formulating seat allocation as a Constraint Satisfaction Problem solved through backtracking search, the system removes manual administrative effort while enforcing accessibility placement and anti-cheating isolation. The project provides a scalable, extensible foundation for a university examination cell.

9. References

[1] Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th Edition), Pearson, 2020.

[2] COMSATS University Islamabad, *CSC262 Artificial Intelligence Lab Manual and Guidelines*, Fall 2026.

[3] Python Software Foundation, *Pandas Documentation: Data Structures and Ingestion Utilities*, https://pandas.pydata.org/